

EEE 495 Electrical and Electronics Engineering Design II

High Level Design Document



Project Title / Product Name: FarmGate Sentinel

Date: November 21, 2025

Team No: 2

Student ID	Name and Surname	e-mail
21802551	Muhammed Gölcük	muhammed.golcuk@ug.bilkent.edu.tr
21903121	Yusuf Semih Gürkök	semih.gurkok@ug.bilkent.edu.tr
22002782	Zeynep İrem İpek	irem.ipek@ug.bilkent.edu.tr
22001648	İsmail Özgenç	ismail.ozgenc@ug.bilkent.edu.tr

Department of Electrical and Electronics Engineering
Bilkent University
Ankara, Türkiye

Contents

List of Abbreviations	2
1 Introduction	3
1.1 Purpose	3
1.2 System Summary	3
1.3 Scope	3
2 Overall System Architecture	3
2.1 Overall Description	3
2.2 Hardware Summary	4
3 Hardware Block Diagram — Edge Module	6
3.1 Configuration Summary	6
3.2 Power Topology and Budget	6
3.3 Storage Footprint for the 5 min Ring Buffer	7
3.4 ESP32-CAM Pad Plan (camera and SD active)	7
4 Hardware Block Diagram — Gateway Module	8
4.1 Configuration Summary	8
4.2 Power Budget	8
4.3 Storage and Throughput	9
4.4 Hardware Interfaces	9
5 Software Part	9
5.1 Edge Software	10
5.2 Gateway Software	11
5.3 Cloud and Mobile Application Software	13
6 Workload Distribution	14
6.1 Overview	14
6.2 Member Responsibilities	15
7 Conclusion	15

List of Abbreviations

Abbreviation	Meaning
ADC	Analog-to-Digital Converter
API	Application Programming Interface
CMOS	Complementary Metal–Oxide–Semiconductor
CSV	Comma-Separated Values
DC	Direct Current
DVP	Digital Video Port
EEPROM	Electrically Erasable Programmable Read-Only Memory
ESP32	Espressif Systems 32-bit Wi-Fi/BLE Microcontroller
GB	Gigabyte
GPIO	General Purpose Input/Output
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
I ² C	Inter-Integrated Circuit Bus
IMU	Inertial Measurement Unit
IP	Ingress Protection (environmental sealing rating)
JSON	JavaScript Object Notation
LTE	Long-Term Evolution (4G Cellular Network)
MCU	Microcontroller Unit
mA	Milliampere
MB	Megabyte
MHz	Megahertz
MQTT	Message Queuing Telemetry Transport
NTP	Network Time Protocol
OTA	Over-The-Air Update
PCB	Printed Circuit Board
PIR	Passive Infrared Sensor
PPP	Point-to-Point Protocol
PSU	Power Supply Unit
PWM	Pulse-Width Modulation
RBAC	Role-Based Access Control
RF	Radio Frequency
RSSI	Received Signal Strength Indicator
SVGA	Super Video Graphics Array (800 × 600 px)
TCP/IP	Transmission Control Protocol / Internet Protocol
TLS	Transport Layer Security
UART	Universal Asynchronous Receiver–Transmitter
UI	User Interface
USB	Universal Serial Bus
WAN	Wide Area Network
WAV	Waveform Audio File Format
Wi-Fi	Wireless Fidelity (IEEE 802.11 b/g/n)

1 Introduction

1.1 Purpose

This High-Level Design Document defines the architecture and principal design decisions for the *FarmGate Sentinel* system. The document is derived from the Technical Specification and maintains traceability to its Functional and Non-Functional Requirements through explicit requirement identifiers that are referenced in the relevant sections.

1.2 System Summary

FarmGate Sentinel operates with camera-equipped edge nodes that perform periodic capture and event-driven burst capture. A gateway aggregates traffic over Wi-Fi, buffers data during wide-area network outages, and forwards data to the cloud using an LTE uplink. Cloud services provide durable storage and alert distribution. A mobile client enables monitoring and actuator control. The design targets timely incident notification and continuous operation under constrained bandwidth and intermittent connectivity as defined in the Technical Specification.

1.3 Scope

- System partitioning across edge nodes, gateway, cloud, and mobile client, including responsibilities of each part.
- High-level hardware and software blocks, typed interfaces, and key data and control flows.
- Performance and capacity budgets that drive design choices, including end-to-end alert latency, throughput, and storage sizing, with direct references to the Technical Specification.

2 Overall System Architecture

2.1 Overall Description

The system consists of camera-enabled edge nodes connected by Wi-Fi to a local gateway, which provides buffering and wide-area connectivity to cloud services. Each edge node performs periodic capture and event-driven burst capture, publishes image frames and metadata to the gateway, and accepts actuator commands. The gateway aggregates traffic from multiple nodes, persists a rolling buffer on removable storage, and forwards data to the cloud over LTE or Ethernet. The cloud layer exposes a lightweight messaging interface for the mobile application, which renders incidents, status, and basic controls.

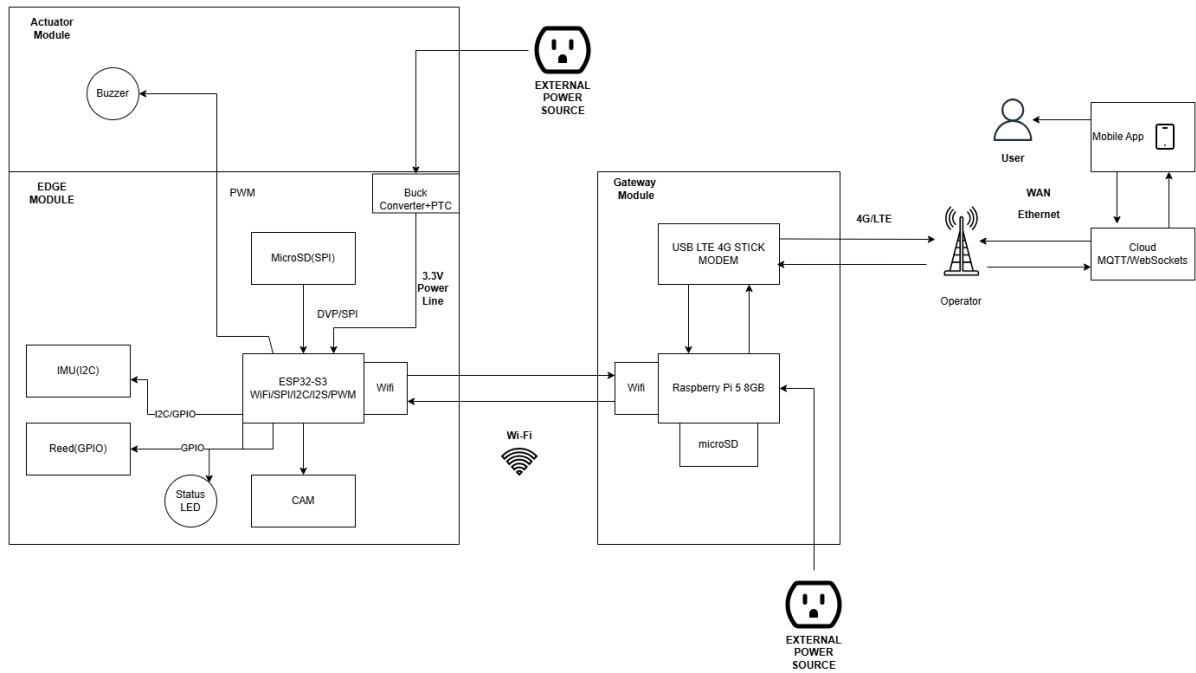


Figure 1: Overall system architecture block diagram.

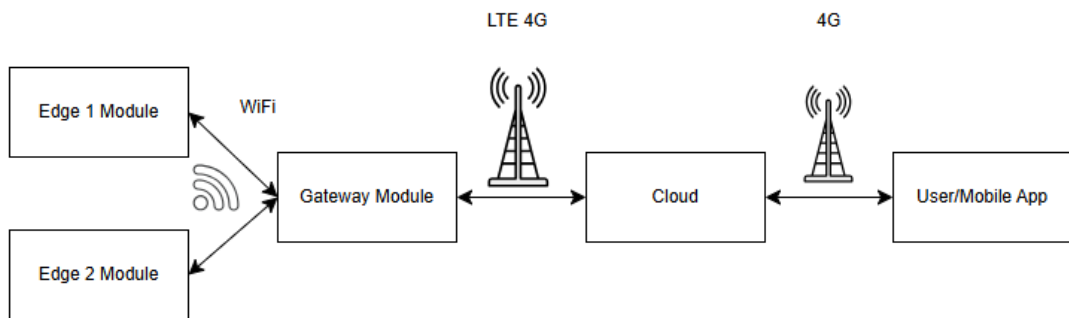


Figure 2: Simplified Overall system architecture block diagram [1].

2.2 Hardware Summary

- **Edge Module.** ESP32-S3 with integrated Wi-Fi manages the camera interface and peripheral sensors[2]. The camera connects via DVP/SPI, the IMU via I²C, the reed switch and status LED via GPIO, the buzzer via a PWM-driven output stage, and optional local storage via SPI microSD. A buck converter with resettable protection provides a regulated 3.3 V rail to the node.
- **Gateway Module.** Raspberry Pi 5 serves as the Wi-Fi aggregation point and runs ingestion, buffering, and forwarding services[3]. Uplink is provided by a USB LTE 4G stick modem; a microSD card supplies local buffer capacity and logs.
- **Backhaul and Cloud.** The gateway transmits incident data and metadata to the cloud over

LTE or Ethernet. The cloud provides an MQTT/WebSocket interface consumed by the mobile application.

- **Mobile Application.** The app displays incidents and node status and issues basic control requests that are relayed to the gateway and edge nodes.
- **Primary Interfaces.** DVP/SPI for the camera, I²C for the IMU, GPIO for the reed switch and status LED, PWM for the buzzer driver, 2.4 GHz Wi-Fi between edge and gateway, and LTE/Ethernet between gateway and cloud.

The ESP32-S3/ESP32-CAM platform was selected for the edge module after evaluating several alternatives, including the Raspberry Pi Zero 2 W and Arduino Portenta H7. While these alternatives offer higher performance, they come with significantly higher power consumption, larger form factor, or greater cost. The ESP32 family provides integrated Wi-Fi, native camera support, low power usage, and a compact footprint, making it well suited for periodic image capture and sensor interfacing. For the camera module, alternatives such as the OV5640 and OV7670 were considered, but the OV2640 was chosen due to its built-in JPEG compression, low memory requirements, and proven compatibility with the ESP32-CAM architecture. At the gateway level, the Raspberry Pi 5 was preferred over options such as the Jetson Nano or Raspberry Pi 4. The Jetson platform, while powerful, is costly and power-hungry for field deployment, and the Raspberry Pi 4 provides insufficient thermal and I/O improvements for running real-time YOLO inference. The Raspberry Pi 5 achieves an optimal balance of processing capability, cost, community support, and power efficiency, enabling reliable AI-based detection and multi-node data aggregation in the system. Also, when sensors and their alternatives examined, it is found that there are no big differences between sensor models for purpose of project. Therefore, models that are easier to access were preferred for sensors.

3 Hardware Block Diagram — Edge Module

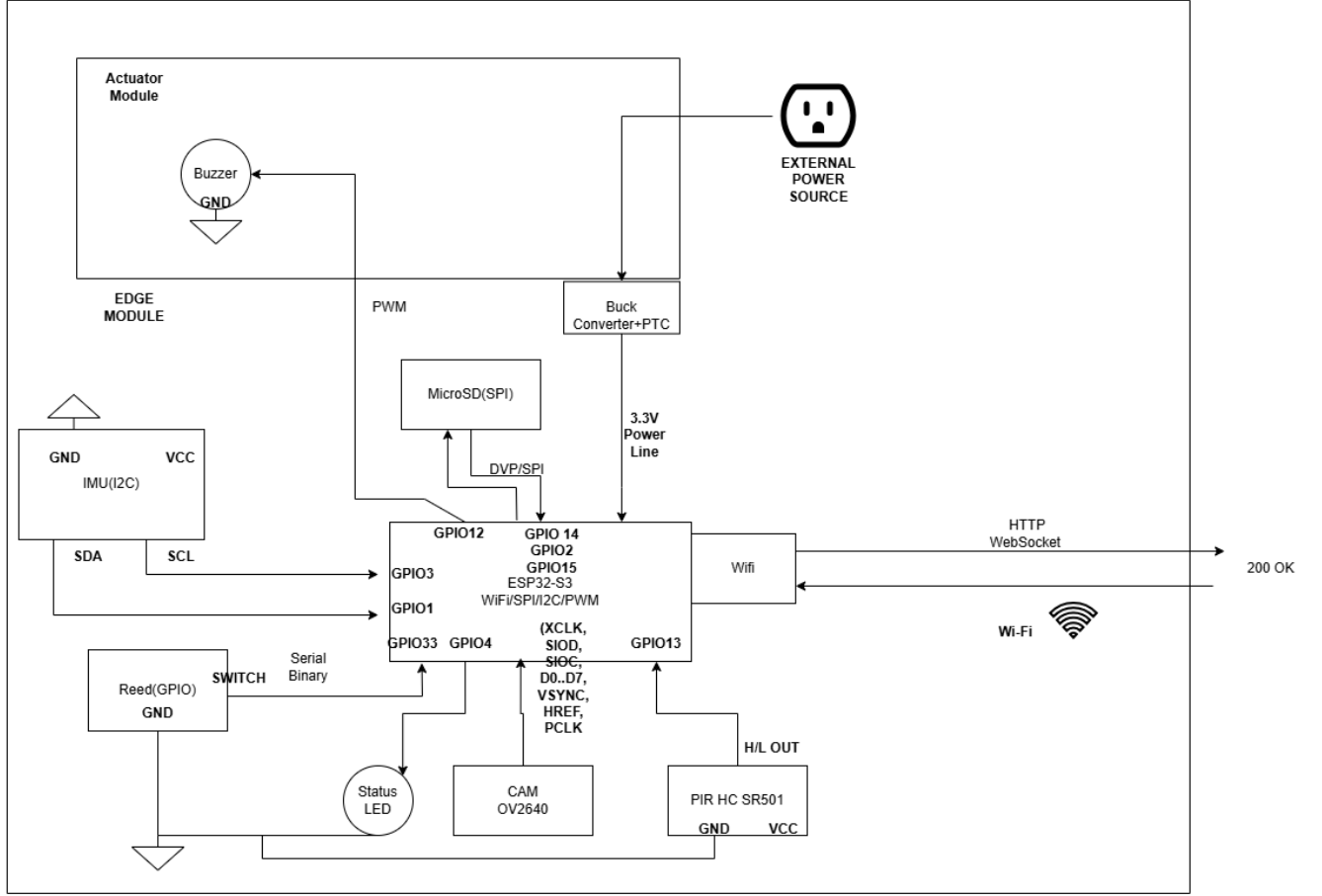


Figure 3: Hardware block diagram of edge module

3.1 Configuration Summary

The edge module uses the ESP32-CAM with the OV2640 camera and on-board microSD. The camera captures one frame every 6 s. Each captured frame is retained on microSD for 5 min as a ring buffer while frames are forwarded over the 2.4 GHz Wi-Fi link to the gateway hotspot. Peripherals on the node are: an I²C IMU at 3.3 V, a PIR motion sensor supplied at 5 V with logic shifted to 3.3 V, a reed-switch contact input, a status LED, and a 3.3 V active buzzer driven by PWM. Power is provided by a USB-C phone charger rated 5 V, 3 A.

3.2 Power Topology and Budget

Input source is 5 V from the charger to the ESP32-CAM 5V pin. The on-board 3.3 V regulator supplies the ESP32, OV2640, and SD, and also feeds the IMU and the 3.3 V buzzer.

$$\begin{aligned}
 I_{\text{ESP32 (avg)}} &\approx 0.12 \text{ A}, & I_{\text{ESP32 (TX peak)}} &\approx 0.24 \text{ A}, \\
 I_{\text{CAM}} &\approx 0.08 \text{ A}, & I_{\text{SD (write)}} &\approx 0.08 \text{ A}, \\
 I_{\text{IMU}} &< 0.005 \text{ A}, & I_{\text{Buzzer}} &\approx 0.02\text{--}0.04 \text{ A}, \\
 I_{\text{LED}} &\approx 0.005 \text{ A}.
 \end{aligned}$$

Average 3.3 V current is 0.25–0.35 A with peaks near 0.5–0.6 A. LDO dissipation at the module is

$$P_{\text{LDO}} = (5.0 - 3.3) \times 0.35 \approx 0.60 \text{ W}.$$

5 V input headroom is ample: $P_{\text{in,peak}} \approx 5.0 \times 0.6 = 3.0 \text{ W} \ll 15 \text{ W}$.

Protection components at the input: resettable fuse $I_{\text{hold}} = 0.75 \text{ A}$, $I_{\text{trip}} = 1.5 \text{ A}$; 5 V TVS diode; bulk decoupling $\geq 2 \times 10 \mu\text{F}$ at 5 V and $\geq 2 \times 10 \mu\text{F} + 2 \times 0.1 \mu\text{F}$ at 3.3 V.

3.3 Storage Footprint for the 5 min Ring Buffer

Frame period $T = 6 \text{ s}$, window $W = 300 \text{ s} \Rightarrow N = W/T = 50$ frames. With mean JPEG size $\bar{s}$ [KB],

$$S_{\text{buffer}} = 50 \cdot \bar{s}.$$

Examples: $\bar{s} = 150 \Rightarrow S \approx 7.5 \text{ MB}$; $\bar{s} = 400 \Rightarrow S \approx 20 \text{ MB}$.

3.4 ESP32-CAM Pad Plan (camera and SD active)

Function	Pad	Notes
Camera DVP bus	Fixed	ESP32-CAM reference mapping; reserved.
SDMMC 1-bit	CLK=GPIO14, CMD=GPIO15, D0=GPIO2	Matches ESP32-CAM hardware; supports required write rate.
I ² C SDA (IMU)	GPIO1 (U0TXD)	Reassigned to SDA after programming.
I ² C SCL (IMU)	GPIO3 (U0RXD)	Reassigned to SCL; external pull-ups to 3.3 V.
PIR input	GPIO13	5 V $\rightarrow$ 3.3 V divider (10k/20k).
Reed input	GPIO33	Digital input with pull-up and RC debounce.
Status LED	GPIO4	On-board flash LED.
Buzzer PWM	GPIO12	Low-side MOSFET gate.

4 Hardware Block Diagram — Gateway Module

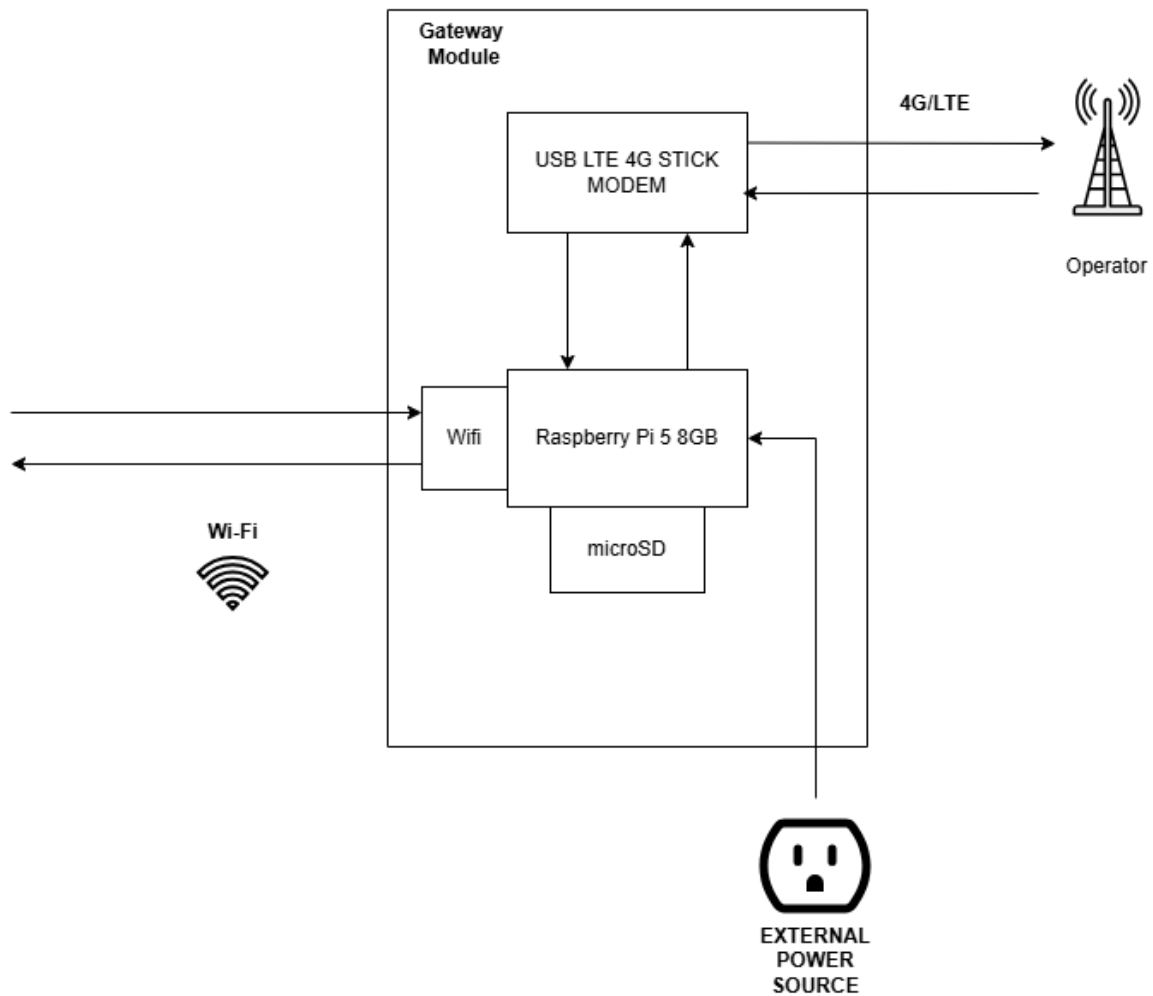


Figure 4: Gateway Hardware Block Diagram

4.1 Configuration Summary

The gateway module is a Raspberry Pi 5 with 8 GB RAM. The module operates the on-board Wi-Fi radio as the local access point. Uplink to the wide area network uses a USB LTE 4G stick modem. Persistent buffering uses the on-board microSD slot. Power input is USB-C at 5 V with a 5 A supply.

4.2 Power Budget

- Raspberry Pi 5 average current is taken as 2.5 A under normal load. Peak current during CPU and GPU bursts is taken as 3.0 A.
- USB LTE stick average current is taken as 0.5 A. Radio attach and transmit peaks are taken as 1.0 A.
- microSD current during sustained writes is taken as 0.10 A.
- On-board Wi-Fi AP current is taken as 0.30 A during traffic.

Peak input current is therefore $I_{\text{peak}} \approx 3.0 + 1.0 + 0.10 + 0.30 = 4.40$ A. Peak input power is $P_{\text{in,peak}} \approx 5.0 \times 4.40 = 22.0$ W. The selected 5 V 5 A USB-C supply covers this envelope.

4.3 Storage and Throughput

Each edge node produces one image every 6 s. The per-node inbound rate is $\bar{s}/6$ bytes per second, where $\bar{s}$ is the mean JPEG size. For N nodes the aggregate inbound rate is $N \times \bar{s}/6$. A buffer of B images requires $S_{\text{buf}} = B \times \bar{s}$ bytes on microSD.

4.4 Hardware Interfaces

- USB-C power input at 5 V.
- USB-A port to the LTE 4G stick modem.
- microSD card slot for local buffering.
- 2.4 GHz on-board Wi-Fi radio configured as access point for edge nodes.

5 Software Part

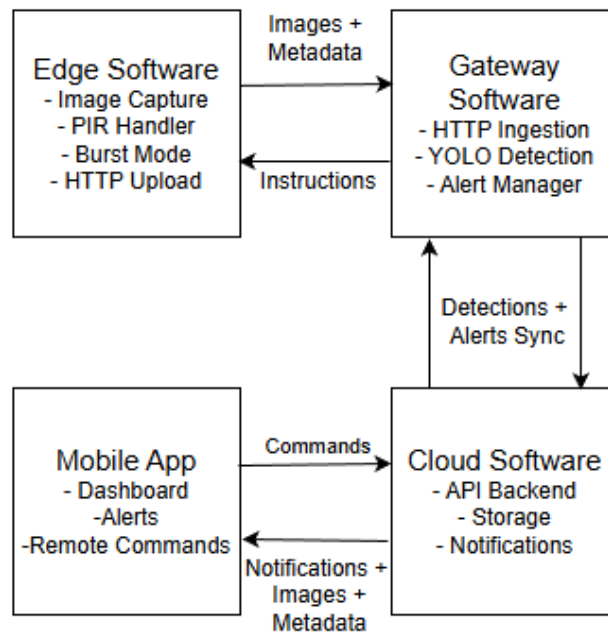


Figure 5: Overall Software block diagram

The software system of the project is designed as a distributed and modular architecture running across multiple hardware layers: edge nodes, a gateway, a cloud service, and a mobile application. Each layer performs a specific subset of operations to ensure reliable detection, data transfer, storage, and user notification. The following subsections present the flowcharts and corresponding descriptions of each software component.

5.1 Edge Software

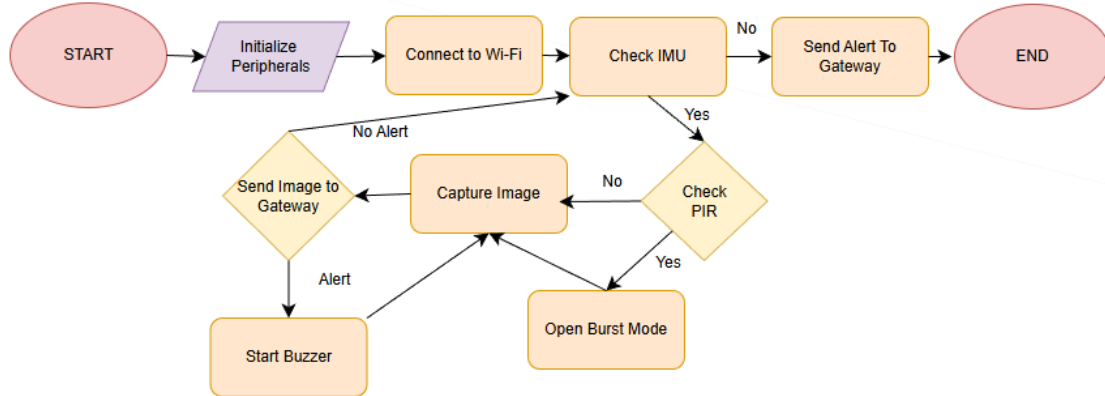


Figure 6: Edge Software Flowchart

The edge software operates on ESP32-CAM modules programmed in C++ using the Arduino framework. Each edge node serves as an autonomous sensing and alert unit responsible for monitoring environmental activity, capturing images, and communicating with the gateway to ensure continuous field surveillance. The operational structure is divided into several subparts to describe its main functional stages.

Initialization: Upon startup, the ESP32 initializes all peripherals including the camera, PIR motion sensor, Reed switch, and Wi-Fi interface. The module then connects to the wireless hotspot generated by the Raspberry Pi gateway. This connection ensures bidirectional communication for both image transfer and alert signaling between the edge node and the gateway.

Reed Switch Monitoring: After establishing connectivity, the system continuously checks the Reed switch status to verify whether the edge node is in place. If the Reed switch indicates an open state, the ESP32 interprets this as the node being displaced or tampered with. In this condition, it immediately sends an alert message to the gateway, which in turn notifies the user through the mobile interface, and the process terminates until the device is repositioned. When the switch remains closed, the node proceeds to normal operation.

Image Capture and Communication: During normal operation, the ESP32 periodically captures images using its camera module and encodes them in JPG format. These images, together with metadata such as node ID and timestamp, are transmitted to the Flask server on the gateway through an HTTP POST request directed to the /upload endpoint. This loop of image acquisition and transmission continues as long as no alert is received from the gateway.

Alert Response: After each upload, the ESP32 awaits a response from the gateway. If the gateway signals an alert condition (for instance, a predator detected by the gateway), the ESP32 immediately activates its buzzer as a deterrence mechanism. Once the buzzer operation completes, the system automatically returns to the image-capture loop to resume normal monitoring. If no alert is issued by the gateway, the node continues capturing and sending images without interruption.

PIR Motion Detection and Burst Mode: Parallel to the regular operation, the ESP32 constantly monitors the PIR motion sensor. If motion is detected within the field of view, the system enters a “burst mode” that increases the frame-per-minute (FPM) rate of image capture. Multiple consecutive images are then transmitted to the gateway to provide higher temporal

resolution for the detection algorithm. After the motion subsides, the system exits burst mode and returns to its regular image-capture cycle.

Overall, this modular and event-driven software structure provides efficient sensing, low latency in alert handling, and effective coordination with the gateway. The firmware employs `WiFi.h`, `HTTPClient.h`, and `esp_camera.h` libraries, combining interrupt-based sensor logic with periodic network communication to balance responsiveness and power efficiency.

The edge node firmware is developed using the Arduino framework for ESP32 and relies entirely on open-source components that permit commercial deployment. Core ESP-IDF/Arduino SDK headers such as `esp_camera.h`, `esp_wifi.h`, `esp_sntp.h`, and `esp_http_server.h` are released under the Apache 2.0 license [4], which allows unrestricted commercial use. Networking and utility libraries including `WiFi.h`, `HTTPClient.h`, `SD_MMC.h`, and `time.h` are also part of the Apache 2.0–licensed ESP32 Arduino core [4]. Image conversion utilities such as `img_converters.h` originate from the ESP-IDF camera driver and are likewise licensed under Apache 2.0 [4]. The C++ Standard Library features used on the edge node (e.g., `vector`, `memory`) follow the GCC libstdc++ GPL runtime exception [5], which permits commercial distribution of compiled firmware. Overall, the edge software stack is fully compliant with commercial licensing requirements and contains no copyleft components that impose disclosure of proprietary code.

5.2 Gateway Software

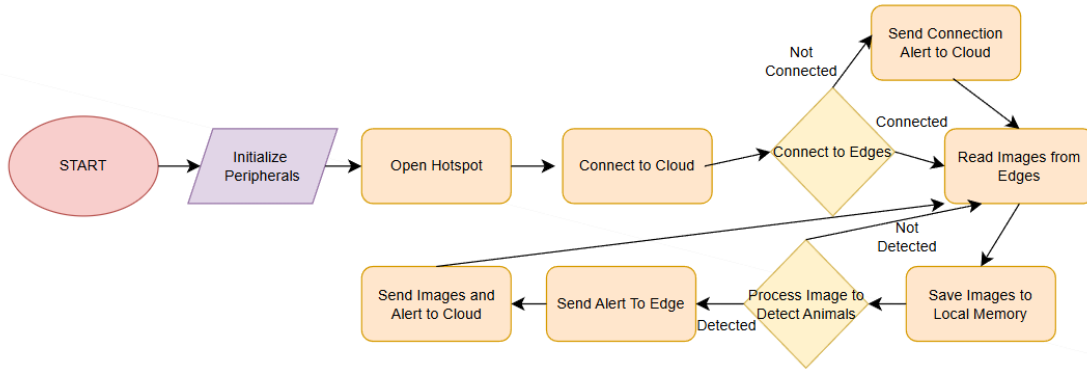


Figure 7: Gateway Software Flowchart

The gateway software runs on a Raspberry Pi 5 using Raspberry Pi OS and is implemented entirely in Python. Acting as the central processing and communication hub of the system, the gateway coordinates between multiple edge nodes, performs real-time AI-based animal detection, manages data storage, and synchronizes with the cloud through an LTE interface. Its software structure can be described through several interconnected functional stages, as illustrated in Figure 8.

Initialization and Connectivity: When powered on, the Raspberry Pi initializes its onboard peripherals and network interfaces. The system first opens a Wi-Fi hotspot, allowing edge nodes to establish a local connection. In parallel, it attempts to connect to the cloud service through the LTE modem to enable remote data synchronization and alert forwarding. If the connection to the cloud fails, a connection alert message is sent to notify the cloud service of the disconnection state, ensuring status awareness of field connectivity.

Edge Communication Management: Once network links are established, the gateway continuously monitors connectivity with all registered edge nodes. If a connection issue with any edge node is detected, a “connection lost” alert is sent to the cloud for monitoring purposes. When communication is active, the gateway starts receiving incoming data streams — including images and metadata — from multiple ESP32-CAM edge nodes through its Flask-based HTTP server.

Image Reception and Processing: After receiving an image, the gateway saves a local copy to its internal storage for traceability. The image is then passed to the YOLOv11-based object detection module running on the Raspberry Pi. The trained model (`best.pt`) processes the image to identify and classify animals such as wolves, bears, wild boars, and other potential threats. The output includes bounding boxes, confidence values, and class labels, which are stored together with timestamps for further analysis.

Alert Generation and Feedback: If the inference result indicates a predator or a security threat, the gateway immediately performs two parallel actions. First, it sends an alert command to the corresponding edge node that captured the image, triggering the buzzer for deterrence. Second, it forwards both the image and alert data to the cloud service for user notification and long-term storage. If no dangerous object is detected, the processed image and its log file are saved locally, and the system continues reading data from other edge nodes.

Cloud Communication and Data Synchronization: The gateway maintains an asynchronous communication link with the cloud storage. All processed data, including detection results, timestamps, and alert statuses, are periodically uploaded to the cloud storage to ensure that the user can access real-time system updates through the mobile application. In case of intermittent LTE disconnections, the data are temporarily cached in the gateway’s local memory and automatically uploaded once the connection is re-established.

This software architecture enables real-time detection, reliable inter-device communication, and consistent data synchronization between the field and the cloud. The main Python modules utilized include Flask for the HTTP server, Ultralytics YOLO for object detection, OpenCV for image preprocessing, and Requests for cloud communication. Multi-threading ensures that the system can concurrently handle image reception, inference, and data transmission without interrupting ongoing connections.

The Raspberry Pi gateway software uses open-source components that are all permitted for commercial use. The YOLOv11 model and the `ultralytics` Python package operate under the AGPL-3.0 license [6], which allows commercial deployment but requires that any modifications to YOLO-related source code be made publicly available. All other libraries in the inference and processing pipeline are permissively licensed for unrestricted commercial use, including PyTorch (BSD-3-Clause) [7], OpenCV-Python (Apache 2.0) [8], Pillow (HPND) [9], `tqdm` (MPL-2.0) [10], and `FiftyOne` (Apache 2.0) [11]. Standard Python modules used for file handling and networking—such as `os`, `pathlib`, `shutil`, and `glob`—are covered by the Python PSF License [12], which also supports commercial applications. Thus, the Raspberry Pi software environment complies fully with commercial licensing requirements, with AGPL obligations applying only to modified YOLO source components.

5.3 Cloud and Mobile Application Software

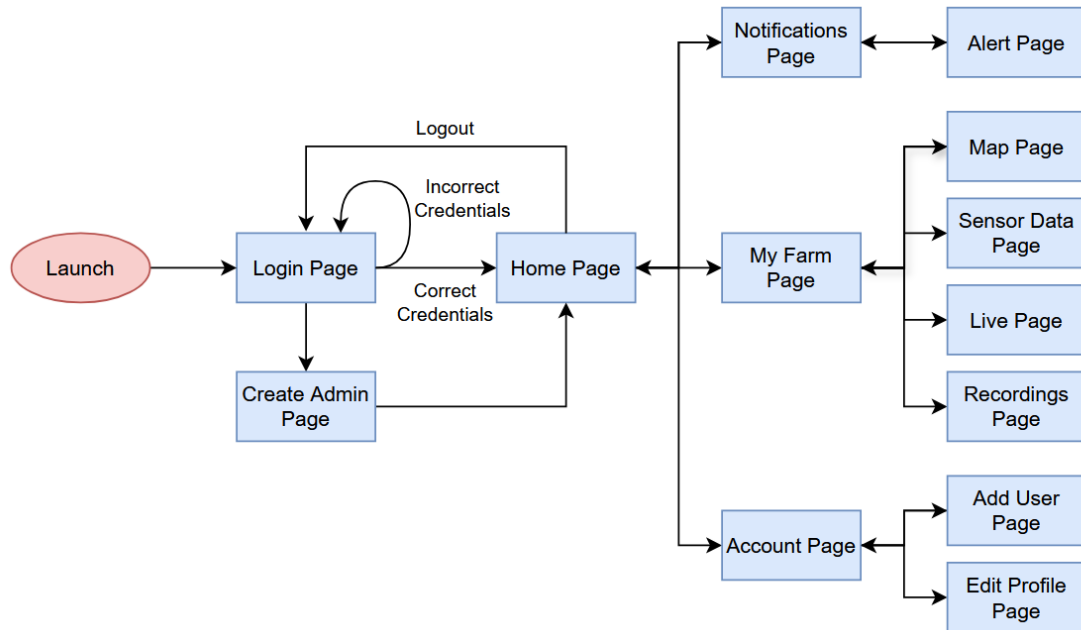


Figure 8: Mobile App Flowchart

The cloud layer functions as the central coordination, storage, and communication hub of the project. In this project, GitHub is used as the cloud platform, enabling the gateway to upload processed detection results, images, and metadata to a remote repository for persistent and accessible storage. The cloud also handles user authentication through secure API endpoints and maintains real-time synchronization, ensuring that all mobile clients receive up-to-date system status, alerts, and historical detection data. Additionally, it relays configuration updates and user-issued commands to the gateway, allowing seamless interaction between the mobile application and field devices. Mobile application is developed using the Flutter framework with the Dart programming language. It serves as the primary user interface for farm owners, enabling real-time monitoring, alert management, and interaction with neighboring systems. Designed with cross-platform compatibility, the application seamlessly operates on Android devices, providing intuitive access to all critical data collected and processed by the system.

System Overview: The mobile application communicates primarily with the cloud API. Upon launch, the app initializes the secure login interface, where user credentials are verified through encrypted communication with the authentication server. Once logged in, the main dashboard page is displayed, showing the current system status, animal detection logs, and live camera feed previews. The data displayed is fetched from the cloud database and periodically updated to maintain real-time accuracy.

User Interface and Navigation Flow: After authentication, the user is directed to the home page, which functions as the central hub for all monitoring and control operations. From this page, users can navigate to several subpages such as the *MyFarm* section, *Account Page*, and *Notification Page*. The *MyFarm* page provides access to recorded detections, live camera requests, sensor data plots, and the farm map interface, which visualizes the spatial positions of all active nodes. The *Account Page* allows users to modify their personal information and, for administrators, register new users under the system. Meanwhile, the *Notification Page* lists

all recent alerts and allows users to view detailed event information or acknowledge received warnings.

Alert Handling and User Actions: When a new detection event occurs—such as the identification of a predator by the YOLO-based model on the gateway—the cloud system immediately sends a push notification to all connected mobile devices of that farm. The alert includes details such as the detected animal type and node location. By selecting the notification, the app redirects the user to the event details page, where annotated images and time stamps are displayed. From this interface, users can initiate deterrent actions remotely, such as activating sound alarms or flashing lights in the corresponding field zone. This two-way interaction enables real-time response to wildlife intrusions.

Additional Functionalities: The upper information bar in the application continuously displays the user’s name, connectivity status. Users can also provide feedback directly to developers through a dedicated feedback interface integrated into the notification section. Furthermore, the app allows configuration of certain operational parameters, such as detection sensitivity, image capture rate, and notification frequency, which are updated on the cloud and synchronized across connected nodes.

Integration and Performance: The mobile application interfaces with the FarmGate backend through RESTful APIs and WebSocket channels. All communications uses HTTPS, and JSON is used for structured data exchange. Local caching mechanisms are used to display the last known system state in case of temporary disconnection.

Overall, the mobile application functions as the user’s operational center for the system, merging real-time field data visualization, cloud synchronization, and remote control capabilities within a secure and accessible interface.

This design ensures end-to-end visibility for farmers and enables distributed monitoring even when away from the field. The mobile application is developed using the Flutter framework and the Dart programming language, both of which are released under permissive licenses suitable for commercial use. The Flutter SDK, including its core UI toolkit and Material Design components, is distributed under the BSD-3-Clause license [13], allowing modification and redistribution without copyleft restrictions. The Dart SDK and its standard libraries (including `io`, `async`, and `convert`) are similarly licensed under the BSD-3-Clause license [14], providing full commercial compatibility. Additional packages such as `path_provider` are published under the MIT license, which is likewise permissive and business-friendly. Android Studio, used for debugging and deployment, is distributed under the Apache License 2.0 with bundled third-party components [15], all of which permit commercial development. Therefore, the mobile software stack imposes no copyleft requirements and is fully compatible with commercial distribution of the FarmGate Sentinel application. However, the application relies on the Google Maps Platform for geographical visualization and location services. This service is consumed via the Maps SDK for Flutter, which operates under the Google Maps Platform Free Tier. This usage is subject to the platform’s standard licensing terms and is strictly limited to 5,000 map loads per day to remain within the no-cost usage allowance.

6 Workload Distribution

6.1 Overview

The project responsibilities are divided among four team members according to their areas of focus: hardware, embedded systems, artificial intelligence and image processing, cloud communication, and mobile application development. Each member contributes to distinct

layers of the overall architecture while maintaining integration through shared interfaces and scheduled testing.

6.2 Member Responsibilities

- **Muhammed Gölcük** — Responsible for the **edge hardware and sensor subsystem design**. He designed the ESP32-CAM-based edge module, integrated the OV2640 camera, PIR motion sensor, IMU, reed switch, and buzzer circuits, and performed the corresponding electrical calculations and pin assignments. Muhammed also implemented the **ESP32-to-Raspberry Pi communication** layer, configuring the HTTP protocol, Wi-Fi link, and data transfer routines between the edge nodes and the gateway.
- **Yusuf Semih Gürkök** — Responsible for the **AI-based image processing subsystem**. He created dataset, trained and optimized the **YOLO Small /YOLOv11 neural network** model for real-time animal detection, integrated the inference engine into the Raspberry Pi gateway, and handled preprocessing (resizing, normalization) and postprocessing (bounding boxes, labeling, confidence output) pipelines. He also benchmarked inference performance and adjusted model parameters to balance accuracy and latency.
- **İsmail Özgenç** — Responsible for the **cloud infrastructure and inter-platform communication**. He developed and maintained the communication between the Raspberry Pi gateway and the cloud server, implemented secure data transfer APIs, and managed the **cloud-to-mobile application synchronization**. His work includes setting up the cloud database, handling data persistence for image uploads and metadata, and designing the REST endpoints for mobile clients.
- **Zeynep İrem İpek** — Responsible for the **mobile application development**. She implemented the user interface and notification system, integrated RESTful APIs for event retrieval and device control, and designed the alert visualization and actuator-control modules. The application allows users to monitor detections, view historical images, and send deterrent commands through an interface built using Flutter.

7 Conclusion

This High-Level Design baselines the FarmGate Sentinel hardware across edge and gateway layers and fixes the quantitative budgets required for implementation. The edge module uses ESP32-CAM with OV2640 and microSD, acquires one frame every 6 s, and maintains a 5 min ring buffer. The corresponding storage requirement is between 7.5 MB and 20 MB for the stated JPEG envelope. The 5 V, 3 A USB-C supply feeds the node, with an average 3.3 V current of 0.25 A to 0.35 A and peaks near 0.5 A to 0.6 A. The resulting LDO dissipation is approximately 0.60 W. Interfaces are fixed as I²C at 3.3 V for the IMU, a level-shifted input for the 5 V PIR output, a pull-up contact input for the reed switch, a 3.3 V LED indicator, and a 3.3 V PWM-driven active buzzer.

The gateway is a Raspberry Pi 5 that operates a local access point, buffers inbound payloads on microSD, and uses LTE or Ethernet for backhaul. The peak input envelope remains within a 5 V, 5 A supply. Aggregate throughput scales with node count as $N \times \bar{s}/6$ bytes per second, which determines buffer sizing and uplink load.

Continuity of field awareness is provided by multi-sensor reporting. Camera obstruction raises an explicit condition, IMU data indicate displacement and fall, PIR maintains motion

awareness during camera outages, and the reed switch reports gate state. These signals are transported together with image metadata, so the user receives the full sensor snapshot even when a current frame is not available.

All allocations, pin plans, and electrical levels in this document are consistent with the stated technical specification and are sufficient to proceed to detailed schematic capture, layout, and firmware pin binding without further architectural change.

References

- [1] Y. S. Gürkök, M. Gölcük, İ. Özgenç, and Z. İ. İpek, *FarmGate Sentinel Product Specification*, EEE495 Design II, Bilkent University, Ankara, Türkiye, 2025.
- [2] Espressif Systems, *ESP32 Series Datasheet*, Version 4.6, Espressif Systems, 2023. [Online]. Available: <https://www.espressif.com/>
- [3] Raspberry Pi Foundation, *Raspberry Pi 5 Product Brief*, Raspberry Pi Ltd., 2023. [Online]. Available: <https://www.raspberrypi.com/>
- [4] Espressif Systems, “ESP-IDF and Arduino-ESP32 License (Apache License 2.0),” 2024. [Online]. Available: <https://github.com/espressif/esp-idf/blob/master/LICENSE>
- [5] Free Software Foundation, “GCC Runtime Library Exception,” 2024. [Online]. Available: <https://www.gnu.org/licenses/gcc-exception-3.1.html>
- [6] Ultralytics, “Ultralytics YOLO License (AGPL-3.0),” 2024. [Online]. Available: <https://github.com/ultralytics/ultralytics/blob/main/LICENSE>
- [7] PyTorch Foundation, “PyTorch License (BSD-3-Clause),” 2023. [Online]. Available: <https://github.com/pytorch/pytorch/blob/main/LICENSE>
- [8] OpenCV Developers, “OpenCV License (Apache License 2.0),” 2023. [Online]. Available: <https://github.com/opencv/opencv/blob/master/LICENSE>
- [9] Pillow Contributors, “Pillow License (HPND),” 2023. [Online]. Available: <https://github.com/python-pillow/Pillow/blob/main/LICENSE>
- [10] tqdm Developers, “tqdm License (MPL-2.0),” 2023. [Online]. Available: <https://github.com/tqdm/tqdm/blob/master/LICENSE>
- [11] Voxel51, “FiftyOne License (Apache License 2.0),” 2024. [Online]. Available: <https://github.com/voxel51/fiftyone/blob/develop/LICENSE>
- [12] Python Software Foundation, “Python PSF License,” 2024. [Online]. Available: <https://docs.python.org/3/license.html>
- [13] Flutter Authors, “Flutter SDK License (BSD-3-Clause),” 2024. [Online]. Available: <https://github.com/flutter/flutter/blob/master/LICENSE>
- [14] Dart Contributors, “Dart SDK License (BSD-3-Clause),” 2024. [Online]. Available: <https://github.com/dart-lang/sdk/blob/master/LICENSE>
- [15] JetBrains/Google, “Android Studio License (Apache License 2.0),” 2024. [Online]. Available: <https://developer.android.com/studio/terms>